

# AI-aided Hidden Camera Detection and Localization based on Raw IoT Network Traffic

Jihyeon Lee  
Culture Technology  
KAIST

Daejeon, Republic of Korea  
jihyeon@kaist.ac.kr

Sangwon Seo  
Electrical Engineering  
Korea University

Seoul, Republic of Korea  
sw\_seo@korea.ac.kr

Taehun Yang  
Computer Engineering  
Chungnam National University

Daejeon, Republic of Korea  
thyang@cclab.cnu.ac.kr

Soochang Park  
Computer Engineering  
Chungbuk National University

Cheongju, Republic of Korea  
cewinter@chungbuk.ac.kr

**Abstract**—This paper proposes a novel scheme to detect and localize the spy cameras based on AI algorithm based raw traffic analytics, named AI-aided Hidden Camera Locator (AHCL). In AHCL, the video streaming data are filtered via the SVM (support vector machine) algorithm to quickly monitor whole raw network traffic from a router to the networks first. Then, gathered traffic data are denoised by the Denoising Autoencoder (DAE) technique to improve the data quality of classification for localization, where a camera transmits video streaming. Based on the proof-of-concept implementation, the proposed scheme can achieve 99.5% positioning accuracy of camera detection with the Ensemble Neural Networks (NNs).

## I. INTRODUCTION

Nowadays in the Internet of Things (IoT) environments, we are exposed everywhere to a huge number of IoT devices, such as smartphones, smart watches, smart tablets, smart cameras, wearable devices, etc. Accordingly, privacy and personal security issues are emerging as high important parts of our life. Especially, these issues have arisen in hotels, motels and houses hosted by the accommodation sharing service [1] [2]. In a rental room from such accommodations, subminiature IoT devices with a camera may be mainly hidden in bedroom, bathroom, etc [3]. Such suspicious IoT devices have facilities of video record/streaming and the Internet access with Wi-Fi. With this, secretly-filmed video data can be recorded/stored in its own device or live-streamed to the Internet. Therefore, a system needs to figure out the presence and location of suspicious devices before privacy and personal security issues happen. Furthermore, it is important to quickly/accurately discover them.

The relevant studies have been proposed to detect and localize IoT devices respectively. They are two-fold: device identification and localization. The device identification detects behaviors of devices, such as data traffic flow and fingerprints [4], [5]. Such technologies for device identification could only detect the existence of devices, not their localization. The localization technology is to decide a place from images generated by one or more cameras [6]–[8]. They mainly are based on video data obtained from huge effort and time to prepare complete video data extracted by analysis of whole data traffic. Since it needs such complete video data, it does not support localization in real-time.

Therefore, this paper proposes AI-aided Hidden Camera Locator (AHCL) based on raw IoT network traffic to detect and localize hidden cameras. AHCL is composed of the monitoring node for gathering raw video data and the edge server for the detection and localization of hidden cameras. The monitoring node firstly gathers network traffic on an ethernet link in real-time. Then, it extracts raw video data from the network traffic through the support vector machine (SVM). And a bunch of raw video data is delivered to the edge server. The edge server conducts progress which refines the raw video data by Denoising Autoencoder (DAE). In this process, in order to extract good-quality image data for accurate localization, obstacles such as moving objects that may interfere with the decision about the hidden camera are removed. With this, the neural network classifier (NNC) which uses the refined data could be a good performance. As a result, we can find a place where a hidden camera is installed.

To evaluate the performance of our AHCL system, it is implemented as the proof-of-concept which consists of the Android smartphone with Zoom application and the desktop PCs with Windows/Linux OS. Experiments are conducted in a laboratory and lecture room instead of accommodations. The smartphone as a hidden camera is used to stream videos about rooms by using the Zoom application. As a monitoring node and edge server, a computer application is developed and installed on desktop PCs. The computer application operates in three steps. It firstly filters the raw video packets of the Zoom application with SVM. Next, it removes the noise of the filtered packet with DAE and classifies the rooms with ensemble neural networks including multi-layer perceptron (MLP), 1D-CNN, and Inception Resnet v2.

The rest of this paper is constructed as follows. The proposed AHCL system is described in section II. Section III describes the data acquisition scheme used in our system. Section IV conducts preliminary experiments to verify the effect on the quality of video images. Then, proof-of-concept experiments prove that our system can be considered an effective scheme in section V. Section VI concludes the paper and provides future work.

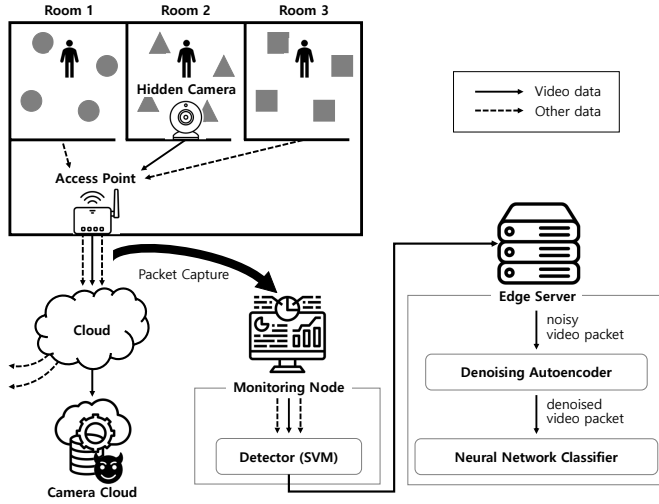


Fig. 1: AHCL System Architecture

## II. AI-AIDED HIDDEN CAMERA LOCATOR (AHCL)

Figure 1 shows the AHCL system architecture in environments where access point and hidden camera are deployed. The proposed system is composed of the monitoring node and edge server. Hidden camera is located in one room and can be connected to the network via the access point. Here, the monitoring node captures packets, which hidden camera generates, delivered from the access point to cloud server on the Internet. It then detects video data generated from the hidden camera, called detector. The video data detected by the detector is delivered to edge server which is composed of denoising autoencoder and neural network classifier. The edge server removes noise (*i.e.*, moving objects) for the video data, and then decides the location (*i.e.*, a room number) of the hidden camera via neural network classifier (NNC) using the denoised data.

We assume that the hidden camera is deployed as follow steps. First, an external person (*i.e.*, invader) installs the hidden camera by stealth. Next, the hidden camera is connected to the access point, which means the invader can manipulate it at the outside of the installed place. Then, the hidden camera takes video which is sent to the cloud by including in packet (*e.g.*, streaming packet). Finally, the camera cloud installed by invader attains the video packet taken by hidden camera.

Meanwhile, the monitoring node observes packets on whether they involve videos, images, or not. When the detector determines packet including the video or image, monitoring node sends this packet to the edge node in order to classify it normal or abnormal. Then, the edge server puts into the received packet to autoencoder to remove noises such as people. After that, the neural network classifier analyzes denoised packet on whether it includes a certain room data or not.

## III. DATA ACQUISITION SCHEME

In our AHCL system, one of the key points is using raw data packets, not complete video data from other schemes,

to detect IoT devices, such as hidden cameras. However, in practice, these raw data packets could not be obtained easily. This is because it has to implement the dedicated application for packet capture corresponding to OS on each desktop PC. For the data packet acquisition in various OS environments, we have developed a computer application for Windows OS and Linux/Mac OS respectively.

In the implemented computer application, the data packet acquisition is performed as following steps: packet reception, packet analysis, and data storage. In the first step, the packet reception is performed. For this, we use open-source libraries for packet capture. For each library, the `pcap_next_ex()` function in WinPcap [9] and the `sniff()` function in Scapy [10] are used. As a result of this step, ethernet frames are continuously received for the running time of the computer application. In the second step, the packet analysis is performed. With this, it extracts data for a video image included in the received packet. For example, in the case of Zoom video, the port number of user data protocol (UDP) uses 8801 when it transmits video data to Zoom media server. Thus, it filters the corresponding port number in the received packets. In the final step, the filtered packet data are stored in CSV format. Then, the stored data are used for machine learning.

## IV. PRELIMINARY EXPERIMENTS

### A. Video Image Denoising

The denoising autoencoder is implemented using MLP, and experiments are conducted using actual video data with a fixed composition. Four 1-minute-long videos shot at different locations are used for training. It captures a video frame by frame and uses the images as input and output of autoencoder. In the video used as input, obstacles such as moving objects are recorded together. That is, the video contains noise. In the video used as the output, only a clean space is recorded without obstructions such as moving objects. To train the denoising autoencoder, data mixed with noise is used as an input, and data without noise is used as the output. An example of actual video data used in the experiment and denoised data can be seen in Figure 2. To test the trained model, video from the same location is used as input. In this case, the video contains obstructions such as moving objects. As a result of checking the output of the denoising autoencoder, it was confirmed that the images captured from each video were correctly denoised into a clean image without any obstructions. Figure 3 is a graph showing the change in the distribution of video data before and after denoising autoencoder training.

### B. Video Capture and Classification

Inception Resnet v2 is used to implement a room classifier, and experiments are conducted using actual video data with a moving composition. For training, two 1-minute-long videos shot at different locations are used, and the video is captured in frame units and used as input to the classifier. The video used as input is a video taken from various angles and heights of space, and only a clean space is recorded without obstructions.

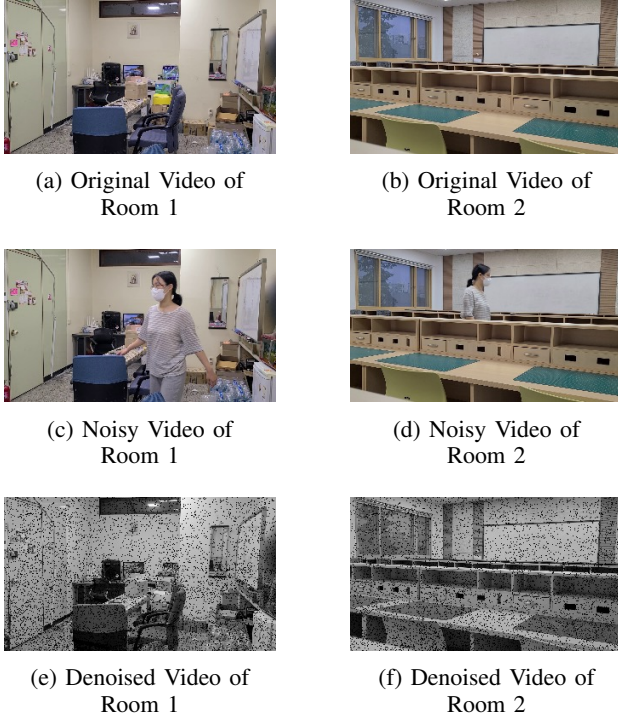


Fig. 2: Video Image Processing

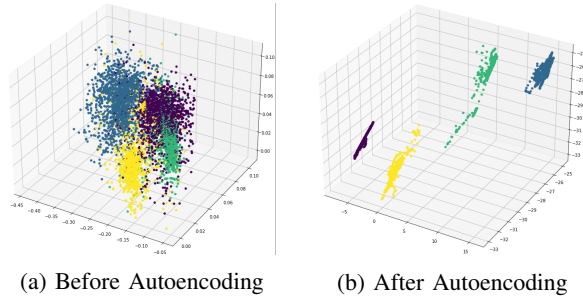


Fig. 3: The distribution of Video Data

As a result of training, the training accuracy is 1.000, and validation accuracy is 0.9932.

## V. PROOF-OF-CONCEPT EXPERIMENTS

### A. Experimental Environments

The data collection environment is as follows. The actual space is filmed in a fixed composition using a camera, and the video is transferred to the outside through Zoom cloud meetings, a video teleconferencing software program. Filming and video transferring take place in three different rooms, and each room is shot from two different compositions. At this time, the packets passing through the router are captured and used for experiments. Packets are captured for 30 minutes for each composition, converted into CSV format, and used as input. The experimental environment for SVM is as follows.



Fig. 4: Classification in a Room

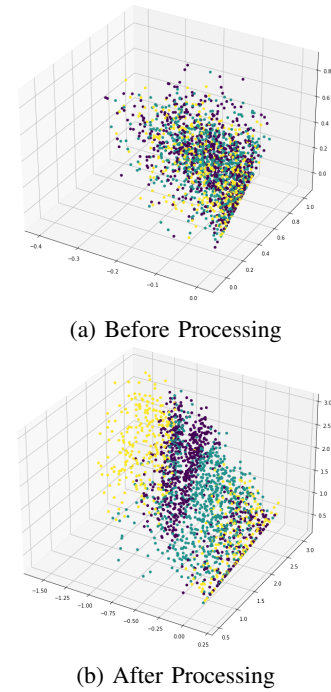


Fig. 5: The distribution of Data

All data collected from the router through which the video packets taken and transferred by the camera pass are used as input to the SVM. Among the input data, different labels are assigned to video packets and non-video packets, and the SVM is trained using these as class labels. The experimental environments for MLP, 1D-CNN, and Inception Resnet v2 are

TABLE I: Accuracy of Room Classification

|                     | Accuracy | Validation Accuracy |
|---------------------|----------|---------------------|
| MLP                 | 0.9995   | 1.0000              |
| 1D-CNN              | 0.9928   | 1.0000              |
| Inception Resnet v2 | 0.9963   | 0.5022              |

as follows. Figure 4 is an example of a video data which are used for streaming at Zoom. These data can be grouped as six of classification (*i.e.*, three spaces with two type of views). Then, Zoom video packets containing these video data are used as input data of each classifier (*i.e.*, MLP, 1D-CNN, Inception Resnet v2). Regardless of the composition, video packet data obtained from the same room are classified into the same class by the classifier. That is, there are three classes. The final prediction is derived by combining the results predicted by each classifier. At this time, the room is classified through the ensemble learning method, and the final class is determined by averaging the probabilities between the prediction results of each classifier through soft voting.

#### B. Video Traffic Detection and Abstraction

Since the collected packet data is data that cannot be linearly separated, the polynomial kernel function and the RBF kernel function, which are kernel methods, can be used for packet classification. And among them, the RBF kernel function, which has faster computation speed and higher accuracy, is used for SVM. As a result of training, it was confirmed that Decision Boundary distinguishes between video packets and non-video packets with an accuracy of 0.9734 for the test data. The average training time of the SVM is 1.4616 seconds, with the minimum required time of 0.0079 seconds and the maximum time of 3.3758 seconds. The average query time of SVM is 0.1733 seconds, with a minimum of 0.0001 seconds and a maximum of 1.0780 seconds.

#### C. Raw Video Traffic Denoising from Cameras

The denoising autoencoder is implemented using MLP. In order to train the denoising autoencoder, the packet is preprocessed in the form of an image and used as input and output. Packet data with noise added is used as input, and clean packet data is used as output. Noise is added by randomly increasing or decreasing the intensity of each pixel by adding a random number between -0.5 and 0.5 to each pixel of the original. Since the result of adding noise could be less than 0 or greater than 1, normalize each pixel intensity between 0 and 1. To check the performance of the denoising autoencoder after training, the video packet data captured in the presence of obstacles such as moving objects is used as input. Changes in data distribution before and after training can be seen in Figure 5. In the graph after training, it can be seen that different rooms are classified better when the data are encoded.

#### D. Room Localization from Video Traffic Analysis

Table I shows the accuracy of classifying the room on MLP, 1D-CNN, and Inception Resnet v2. The video packet data is

pre-processed to form a two-dimensional image and used as an input to the classifier. MLP implements a model with one hidden layer. As a result of training, the room is classified with an accuracy of 0.9995 and a validation accuracy of 1.0000 is shown. 1D-CNN implements a model with one hidden layer. As a result of training, the room is classified with an accuracy of 0.9928 and a validation accuracy of 1.0000 is shown. The training result of Inception Resnet v2 shows an accuracy of 0.9963, but a validation accuracy of 0.5022.

## VI. CONCLUSION AND FUTURE WORK

This paper proposes AHCL system that discloses the hidden camera to preserve privacy. AHCL system is based on AI algorithms filtering the video packets, denoising to improve the classification accuracy, and classifying where the device is located in. Conventional schemes use the complete video data. However, AI algorithms in our system uses the raw video packet without any data processing for neural network. Therefore, it can swiftly find the video packet and determine the location of hidden camera. Our system is implemented as the proof-of-concept. The performance of our system achieves 99.5% of accuracy. For future work, we will investigate the hidden camera localization algorithm with various environments.

## REFERENCES

- [1] The Atlantic, Airbnb Has a Hidden-Camera Problem, USA, Apr., 2022. [Online]. Available: <https://www.theatlantic.com/technology/archive/2019/03/what-happens-when-you-find-cameras-your-airbnb/585007/>
- [2] BBC, Hotel guests 'secretly filmed and live-streamed', UK, Apr., 2022. [Online]. Available: <https://www.bbc.com/news/technology-47638919>
- [3] The Washington Post, Does Your Airbnb or Hotel Have a Hidden Camera? Experts share tips for protecting yourself, USA, Apr., 2022. [Online]. Available: <https://www.washingtonpost.com/travel/tips/airbnb-hidden-camera-tiktok/>
- [4] S. A. Hamad, W. E. Zhang, Q. Z. Sheng, and S. Nepal, "IoT Device Identification via Network-Flow Based Fingerprinting and Learning," in *Proc. of IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering*, Rotorua, New Zealand, Aug. 2019.
- [5] B. Charyyev, and M. H. Gunes, "Locality-Sensitive IoT Network Traffic Fingerprinting for Device Identification," *IEEE Internet of Things Journal*, vol. 8, no. 3, pp. 1272-1281, Feb. 2021.
- [6] N. Tran, D.-K. L. Tan, A.-D. Doan, T.-T. Do, T.-A. Bui, M. Tan, and N.-M. Cheung, "On-Device Scalable Image-Based Localization via Prioritized Cascade Search and Fast One-Many RANSAC," *IEEE Transactions on Image Processing*, vol. 28, no. 4, pp. 1675-1690, Apr. 2019.
- [7] D. Liu, Y. Cui, X. Guo, W. Ding, B. Yang and Y. Chen, "Visual Localization for Autonomous Driving: Mapping the Accurate Location in the City Maze," in *Proc. 2020 25th International Conference on Pattern Recognition (ICPR)*, Milan, Italy, Jan. 2021.
- [8] B. Fan, Y. Yang, W. Feng, F. Wu, J. Lu and H. Liu, "Seeing through Darkness: Visual Localization at Night via Weakly Supervised Learning of Domain Invariant Features," *IEEE Transactions on Multimedia*, early access, Feb. 2022.
- [9] WinPcap 4.1.2 Documentation, WinPcap, 2021. [Online]. Available: [https://www.winpcap.org/docs/docs\\_412/html/main.html](https://www.winpcap.org/docs/docs_412/html/main.html)
- [10] Scapy 2.4.5 Documentation, Scapy, 2021. [Online]. Available: <https://scapy.readthedocs.io/en/latest/usage.html>